

Select Multiple Choice Questions

```
In [ ]: values = [1,1]
        for i in range(3):
            values.append(values[i-1] + values[i-2])
        print(values)
```

```
In [ ]: letters = ["a", "b", "c", "d", "e"]
        print(letters[len(letters)-4])
```

```
In [ ]: x = 21
        if x-5 > 15 :
            print('True')
        if x >= 21 :
            print('True')
        if 3**3 < x + 10 :
            print('True')
        if x != 20 :
            print('True')
```

```
In [ ]: price = 1000
        if price > 1500:
            discount = 0.1 * price
        elif price > 1000:
            discount = 0.05 * price
        else:
            discount = 0.01 * price
        print(discount)
```

```
In [ ]: expression1 = True
        expression2 = False
        print((expression1 and expression2) or expression2)
```

```
In [ ]: a = 0
        for b in range(0,7,2):
            a += b
        print(a)
```

```
In [ ]: n = 0
        while n <= 15:
            n = n+5
        print(n)
```

```
In [ ]: phrase = "The black and the gold"
        if len(phrase) < 15 or phrase[1] == "T":
            print(phrase[-4:])
        else:
            print(phrase[-1] + phrase[-3:-5:-1])
```

```
In [ ]: number_dictionary = {1: "one", 0: "a", 3: "three", "three": 0}
print(number_dictionary[0] * 3)
```

```
In [ ]: def fun(x, ans = 0):
        if(x==1):
            return 1
        else:
            return (x * ans)
print(fun(2,1))
```

```
In [ ]: scale = "doremifasolatio"
new_scale = "" #empty string
new_scale += scale[8:10]
new_scale *= 2
print(new_scale)
```

```
In [ ]: x = do_something(y, z)
```

```
In [ ]: my_sum = 0
number = 1
while my_sum < 10:
    if number < 5:
        number += 3
        continue
    elif number <= 10:
        my_sum += number
    else:
        break
print(my_sum)
```

Coding Section - note that not every cell needs to print. You won't be penalized for printing unless it greatly affects the code (infinite loop, etc.)

Cell 1

```
In [1]: r = "23.31 24.23 -19.44 26.89 16.26 28.88 -6.24 19.42 9.54 -0.73 11.39 29.60 13.41"
```

Cell 2

```
In [4]: returns = r.split()
print(returns)
```

```
['23.31', '24.23', '-19.44', '26.89', '16.26', '28.88', '-6.24', '19.42', '9.54', '-0.73', '11.39', '29.60', '13.41', '0.00', '12.78', '23.45', '-38.49', '3.53', '13.62', '3.00', '8.99', '26.38', '-23.37', '-13.04', '-10.14', '19.53', '26.67']
```

Cell 3

```
In [5]: returns = [float(x) for x in returns]
print(returns)
```

[23.31, 24.23, -19.44, 26.89, 16.26, 28.88, -6.24, 19.42, 9.54, -0.73, 11.39, 29.6, 13.41, 0.0, 12.78, 23.45, -38.49, 3.53, 13.62, 3.0, 8.99, 26.38, -23.37, -13.04, -10.14, 19.53, 26.67]

Cell 4

```
In [16]: sp = sum(returns)/len(returns)/100
          print(sp)
```

0.08497407407407406

Cell 5

```
In [7]: s = "2000 2000 2000 2000 3000 3000 3000 4000 4000 4000 5000 5000 5000 5000 5500 5500"
```

Cell 6

```
In [8]: limits = [float(x) for x in s.split()]
```

Cell 7

```
In [23]: avg_interval = round((max(limits)-min(limits))/len(limits))
          print(avg_interval)
```

185

Cell 8

```
In [24]: interval = round(500/avg_interval)
```

Cell 9

```
In [25]: print("sp:",sp)
          print("interval:",interval)
```

```
sp: 0.08497407407407406
interval: 3
```

Question 2

Cell 1

```
In [17]: def roth_simulator(percent_return, interval, contribution=7000, years=35):
          catch_up = 0
          balance = 0
          for i in range(years):
              if (i+2)%interval == 0:
                  contribution += 500
              if i > 24:
```

```

        catch_up = 1000
        balance += contribution
        balance += catch_up
        balance *= (1+percent_return)
        print(balance)
    return balance

```

Cell 2

```
In [18]: bal = roth_simulator(sp, interval)
```

```

7594.818518518518
16377.486745445814
25906.45407285605
36245.13657579508
48004.82608860421
60763.7843291629
74606.92323236285
90168.85708317804
107053.15185376824
125372.17393888308
145790.3250006601
167943.4895432162
191979.0987305866
218599.59859049713
247482.1507773926
278818.97109326563
313361.09573694266
350838.4054387674
391500.314831292
436159.91936156736
484614.43243531714
537186.322892219
594767.9481000566
657242.5185936107
725025.80786803
800196.3804973493
881754.502933463
970242.9513068054
1066793.110683961
1171547.5304558948
1285203.3600531155
1409059.4755705178
1543440.149822423
1689239.6974423337
1847970.913658702

```

Cell 3

```
In [21]: print(bal)
#Note that you did not need to format this in any specific way. Normally you would
```

```
1847970.913658702
```

AI Version of the Code:

```
In [22]: returns = [23.31,24.23,-19.44,26.89,16.26,28.88,-6.24,19.42,9.54,-0.73,11.39,29.60,
sp = sum(returns) / len(returns) / 100

limits = [2000,2000,2000,2000,3000,3000,3000,4000,4000,4000,5000,5000,5000,5000,550
avg_increase = (max(limits) - min(limits)) / len(limits)
interval = round(500 / avg_increase)

def roth_simulator(yearly_return, interval, contribution=7000, years=35):
    catch_up = 0
    balance = 0
    for x in range(years):
        if (x+2) % interval == 0:
            contribution += 500
        if x > 24:
            catch_up = 1000
        balance += contribution
        balance += catch_up
        balance = balance * (1 + yearly_return)
    return balance

bal = roth_simulator(sp, interval)
print("sp =", sp)
print("interval =", interval)
print("Final balance = $", round(bal, 2))

sp = 0.08497407407407406
interval = 3
Final balance = $ 1847970.91
```

In []: